Commitment Before Appearance

Event Histories, Material State, and the Two Folds of a Layered World

Flyxion
Independent Researcher

September 2026

Abstract

Layered systems make difficult mechanisms locally invisible by stabilizing them behind interfaces. That account, developed in *Order and Obfuscation*, explains how abstraction relocates complexity but leaves open a further question: what exactly is related when a system distinguishes its appearance, its present state, and its history?[1] This essay argues for a qualified dual model. An append-only event history is authoritative for identity, provenance, and the admissibility of later commitments, while a cached material state is authoritative for efficient rendering at a given instant. The state and the semantic record are not successive visual layers, nor does rendering repeatedly inspect the entire history. They are two folds over one committed log: a physical fold that maintains the current operational state and a semantic fold that preserves what the same events mean for provenance, refusal, dependency, and admissible continuation. Appearance is then a disposable projection of current state under transient viewing conditions. Mere looking does not alter history. Observation becomes an event only when it is attested and thereby constrains what may subsequently count as an admissible world. This separation retains the authority of history without confusing archival authority with runtime access, and it resolves the apparent conflict between history-first identity and state-first performance.

1 The Unresolved Question Beneath Layering

Order and Obfuscation describes abstraction as the disciplined relocation of difficulty across interfaces [4]. A higher layer becomes tractable because lower-level obligations have been executed, encapsulated, or made conditional. The account is deliberately opposed to the idea that abstraction annihilates complexity. What disappears from view persists as implementation, assumption, deferred maintenance, or exposure risk.

That argument is sufficient when the layers are compiler, runtime, operating system, and hardware. It is less decisive when the proposed layers are appearance, state, and history. These terms tempt a simple stack:

$$H_{\leq t} \longrightarrow M_t \longrightarrow x_t,$$

[1] A companion essay, *Layering the World: History, State, and Appearance in Persistent Generative Worlds*, treats the same question as a three-layer stack (appearance, operative state, committed history) grounded in the learned-world-models literature. The present essay arrives at a closely related but distinct construction: rather than a stack, it treats the material and semantic records as two parallel folds of one committed log. Readers interested in the persistent-generative-worlds application, the typed event vocabulary, and the distributed-history treatment should consult that essay; the present one is the more general, implementation-agnostic statement of the underlying algebra.

where a history produces a present material state and that state produces a visible appearance. The diagram is useful but ambiguous. It may be read causally, computationally, epistemically, or ontologically. Worse, it suggests that history is merely a deeper state continuously consulted by the layers above it. That interpretation makes an append-only log the hidden implementation of every frame and turns every act of viewing into a possible historical mutation.

The correction is to distinguish authority from access. History may be authoritative for a question without being the object consulted in every operation. A legal archive is authoritative for title without being reread whenever a door opens. A source repository is authoritative for provenance without being replayed whenever a binary draws a pixel. Conversely, the current executable state may be authoritative for what can be rendered now without becoming authoritative for how that state came to exist.

The layering question is therefore not solved by choosing either state or history as the single fundamental object. It is solved by assigning each a different jurisdiction.

2 Three Objects and Three Kinds of Authority

Let

$$H_t = (e_1, e_2, \ldots, e_t)$$

be the committed event history through time t. Let M_t be the cached material or operational state. Let $x_{t,\omega}$ be an appearance produced under transient viewing conditions ω, which may include viewpoint, display parameters, sampling choices, level of detail, or an observer-specific query.

These objects answer different questions. History answers how the world acquired its commitments and which distinctions remain recoverable. Material state answers what operational configuration is presently available. Appearance answers what a particular rendering procedure exposes under a particular set of viewing conditions. Their authority is task-relative:

$$H_t \quad \text{is authoritative for identity and provenance,} \tag{1}$$

$$M_t \quad \text{is authoritative for current operational rendering,} \tag{2}$$

$$x_{t,\omega} \quad \text{is authoritative only for the delivered appearance.} \tag{3}$$

This division prevents two symmetric errors. Snapshot reductionism treats M_t as a sufficient replacement for the path that produced it. Log absolutism treats H_t as the data structure from which every present operation must be recomputed. The first destroys historical distinctions; the second confuses normative authority with an inefficient access pattern.

Definition 1 (Historical authority)**.** *A history H_t is authoritative for a property P when disputes about P are settled by the committed events and their ordering, rather than by any lossy state derived from them.*

Definition 2 (Operational authority)**.** *A material state M_t is authoritative for an operation Q when the system is entitled to execute Q against M_t without replaying H_t, provided that M_t is certified as a valid fold of the relevant committed history.*

The second definition is what makes the model qualified rather than naively history-first. A cache is not epistemically inferior merely because it is derived. Its authority is narrower. It can govern rendering and interaction while remaining revisable or reconstructible in light of the log.

3 The Event Log Is Not a Visual Layer

An event log is often represented below state in an architectural diagram, but "below" is misleading. The relation is not spatial containment. The log and the state need not resemble one another, share a schema, or live in a common file tree. A sequence of commitments may be stored as immutable records; the current state may be a graph, an index, a spatial field, a database image, or a collection of runtime objects. Their relation is given by an update rule, not by structural similarity.

Let the material update algebra be

$$U_M : \mathcal{M} \times \mathcal{E} \to \mathcal{M}.$$

Starting from an initial material state M_0, the current state is

$$M_t = \mathrm{Fold}_M(H_t; M_0) = U_M(\cdots U_M(U_M(M_0, e_1), e_2) \cdots, e_t).$$

In an actual implementation, M_t is maintained incrementally. The formula specifies legitimacy, not a requirement to scan the entire log on every use. Checkpoints, snapshots, indexes, and incremental views are admissible because they preserve the fold relation. Distributed systems have long separated an ordered history from snapshots that summarize a cut through execution [6, 3]. Database recovery similarly distinguishes durable history from a current image optimized for ordinary access [5].

Rendering then has the narrower form

$$x_{t,\omega} = \mathrm{Render}(M_t, \omega_t),$$

not

$$x_{t,\omega} = \mathrm{Render}(M_t, H_{\le t}, \omega_t).$$

The latter expression grants the renderer unnecessary archival jurisdiction. It also hides an unbounded computation inside what should normally be disposable surface work. If provenance must be displayed, the relevant certificate or semantic projection can be made part of the cached state supplied to the renderer. The rendering function still need not interpret the raw history.

4 One Log, Two Folds

The present material state is not the only derived object. The same committed events also support a semantic fold. Let

$$U_S : \mathcal{S} \times \mathcal{E} \to \mathcal{S}$$

update a semantic record S_t containing such matters as event identity, provenance, refusal, dependency, custody, and constraints on later admissibility. Then

$$S_t = \mathrm{Fold}_S(H_t; S_0).$$

The architecture is therefore not one long pipeline from history to picture. It contains two distinct reductions of the same source:

$$H_t \mapsto M_t, \qquad H_t \mapsto S_t, \qquad (M_t, \omega_t) \mapsto x_{t,\omega}.$$

The physical fold answers, "What configuration has these commitments produced?" The semantic fold answers, "What do these commitments establish, preserve, forbid, or leave unresolved?" The folds may coincide for simple events, but no general requirement makes them isomorphic. A refusal, for example, may leave the visible material state unchanged while altering the semantic record of admissibility. Two histories may yield indistinguishable rendered states while preserving different chains of custody. Conversely, several appearances may be rendered from one material state without generating any new commitment at all.

This two-fold construction clarifies the sense in which history remains authoritative. Historical authority resides in the ability to regenerate, audit, or adjudicate both folds and to detect when a cached projection has departed from the committed source. It does not require the log to participate directly in every frame.

Principle 1 (Dual-fold authority)**.** *For a committed history H_t, operational behavior should be mediated by a validated material fold M_t, while identity, provenance, and admissibility should be adjudicated by a semantic fold S_t or, when necessary, by the underlying history. Neither fold may silently substitute for the other.*

This principle also supplies a precise meaning for cache validity. If C_t is a checkpoint at event position $k \leq t$, then

$$M_t = \mathrm{Fold}_M((e_{k+1}, \ldots, e_t); C_t)$$

is legitimate when C_t is certified to equal $\mathrm{Fold}_M(H_k; M_0)$. State can therefore be fast without becoming historically ungrounded.

5 Looking Is Not an Event

The word *observation* causes a second confusion. In ordinary graphics and measurement systems, observation means sampling or rendering. In epistemic and legal settings, observation may mean registering evidence in a way that changes what later claims remain defensible. These are not the same operation.

Suppose a renderer produces

$$x_{t,\omega} = \mathrm{Render}(M_t, \omega_t).$$

Changing ω_t may rotate a camera, filter a signal, inspect a detail, or run a diagnostic. These computations are transient. Unless their results are committed, they do not extend H_t. If every look appended an event, historical growth would depend on observer count, refresh rate, debugging activity, and user-interface design. Two otherwise identical worlds would acquire different identities merely because one was watched more often.

The necessary boundary is attestation. Let o be a rendered or measured observation, and let a specify the agent, method, scope, and asserted evidential consequence. Then

$$e_{t+1} = \mathrm{Attest}(o, a)$$

is a candidate event only when it is accepted under the world's commit rule. Once committed, it may narrow the admissible continuation set:

$$\mathcal{A}_{t+1} = \{c \in \mathcal{A}_t : c \text{ is compatible with } e_{t+1}\}.$$

The observation matters historically because it has been made answerable, not because photons reached an eye or values reached a screen.

Principle 2 (Attestation boundary)**.** *Rendering, sampling, and inspection are disposable computations. An observation becomes a historical event only when it is attested and accepted as a constraint on subsequent admissibility.*

This distinction parallels established separations between an occurrence and a durable record of that occurrence. Lamport's logical clocks order events that processes record; they do not turn every possible perception into a system event [6]. In scientific data management, provenance concerns the accountable derivation and custody of results, not the mere fact that a result was displayed [2, 7]. The attestation boundary applies the same discipline to layered worlds.

Attestation should not be mistaken for infallibility. A committed report may later be challenged, contextualized, or superseded. Its historical identity remains the identity of a particular claim made under particular conditions. Later evidence changes the admissible interpretation of that claim without rewriting the fact that it was made. This is precisely why attestation belongs in the append-only record while rendering does not.

6 Same Appearance, Different World

The insufficiency of snapshots can be stated formally. Define visual equivalence under rendering rule R by

$$M \sim_R M' \quad \Longleftrightarrow \quad R(M, \omega) = R(M', \omega)$$

for the relevant class of viewing conditions. Define historical equivalence for a semantic task T by

$$H \equiv_T H' \quad \Longleftrightarrow \quad T(\mathrm{Fold}_S(H)) = T(\mathrm{Fold}_S(H')).$$

In general,

$$M_t \sim_R M'_t \not\Rightarrow H_t \equiv_T H'_t.$$

Two terminal states may look the same while differing in authorization, provenance, rejected alternatives, or permitted futures. A file recreated byte-for-byte after deletion is visually and extensionally identical to its predecessor, yet it may not possess the same custody chain. Two objects placed at the same coordinates may differ because one arrived through an admissible transition and the other through an unauthorized rewrite. A refusal that leaves an option represented can be materially silent while semantically decisive.

The reverse separation also matters. Historical difference does not imply a visible difference under every rendering. A display is a quotient: it intentionally identifies states or histories that its rule does not distinguish. This is not a defect. It becomes a defect only when a quotient optimized for appearance is later treated as sufficient evidence for identity.

The two-fold model therefore makes a principled place for loss. Material caches may discard historical detail; renderings may discard material detail; semantic projections may discard facts irrelevant to a specified inquiry. What they may not do is erase the existence of the more discriminating source and then claim universal authority.

7 Layering Reconsidered

The earlier displacement thesis can now be refined. An abstraction boundary does not merely conceal a lower layer. It establishes a task-specific quotient and transfers authority over a limited

class of operations to the resulting surface. Complexity is relocated partly because distinctions judged irrelevant to those operations are withheld.

In the present model, the material fold is a controlled reduction of committed history for operational use. Rendering is a further controlled reduction of material state for appearance. The semantic fold is not simply another point on that same descending chain. It runs alongside the material fold because it preserves a different family of distinctions. A provenance query and a frame render are not deeper and shallower answers to one question; they are answers to different questions derived from a common commitment base.

This resolves the tension between the file tree and the conceptual model. The files need not be linked by containment, and the runtime objects need not expose their historical representation. The linkage is algebraic and evidential: both folds claim descent from the same ordered events. Their integrity can be checked by replay, digests, version positions, invariants, or correspondence proofs. Abadi and Lamport's account of refinement mappings captures the broader formal idea that a lower-level execution may justify a higher-level specification without the two sharing a literal representation [1].

The architecture also changes how technical debt should be understood. In *Order and Obfuscation*, debt is the return of structure concealed beneath a once-stable interface. Here a second failure mode appears: jurisdictional drift. A material cache begins answering provenance questions; a renderer becomes a commit mechanism; a semantic index is treated as an operational state; or a log is placed on the critical path of disposable presentation. The problem is not simply that a layer leaks. It is that one projection is asked to exercise authority outside the task for which it was constructed.

8 Design Consequences

A system built on the qualified dual model should maintain a small set of separations. Commit operations append to H_t and advance both folds. Material updates may be transactional or incremental, but they must be traceable to committed event positions. Semantic updates preserve distinctions required for identity, provenance, and admissibility even when those distinctions have no current visual expression. Render operations read M_t and transient parameters; they do not write history unless an explicit attestation operation follows. Checkpoints accelerate reconstruction but identify the history prefix they summarize. Reconciliation procedures compare cached folds against the log without treating ordinary access as replay.

These are not merely implementation preferences. They protect the conceptual boundaries on which the model depends. If rendering itself commits, observation volume changes world identity. If snapshots decide provenance, distinct histories collapse prematurely. If every query scans raw history, archival authority becomes an obstacle to operation. If semantic records can mutate without corresponding commitments, the system acquires an unofficial history alongside the official one.

The desired invariant is therefore:

$$\begin{aligned} M_t &= \mathrm{Fold}_M(H_t; M_0), \\ S_t &= \mathrm{Fold}_S(H_t; S_0), \\ x_{t,\omega} &= \mathrm{Render}(M_t, \omega_t), \\ H_{t+1} &= H_t \parallel e_{t+1} \quad \text{only after a valid commitment.} \end{aligned}$$

The equations state logical relationships. They leave open whether folds are eager or lazy, centralized or distributed, cached in one structure or many, deterministic or explicitly parameterized. What matters is that the jurisdictions remain legible.

9 Conclusion

Layering does not require a choice between history and state. It requires a disciplined account of what each is allowed to settle. History is authoritative for identity, provenance, and the sequence of commitments that constrains the future. Cached material state is authoritative for efficient present operation and rendering, so long as it remains a valid fold of that history. Appearance is a transient projection, not a new ontological layer and not a commit trigger.

The decisive structure is thus one log and two folds. The physical fold maintains what the world can presently do. The semantic fold maintains what the world is entitled to claim about how it became so. Rendering consults the first; adjudication consults the second or returns to the log. Attestation is the bridge by which a disposable observation becomes a durable constraint.

This refinement preserves the central insight of *Order and Obfuscation*: abstraction creates usable surfaces by relocating complexity. It adds that surfaces do not inherit unlimited authority from their usefulness. A stable image may conceal several material states; a stable state may conceal several histories; and a history may support several legitimate projections. The work of architecture is not to force these into one representation. It is to preserve their distinct jurisdictions while keeping their common commitments recoverable.

References

[1] M. Abadi and L. Lamport. The existence of refinement mappings. *Theoretical Computer Science*, 82(2):253–284, 1991.

[2] P. Buneman, S. Khanna, and W.-C. Tan. Why and where: A characterization of data provenance. In *Database Theory—ICDT 2001*, pages 316–330. Springer, 2001.

[3] K. M. Chandy and L. Lamport. Distributed snapshots: Determining global states of distributed systems. *ACM Transactions on Computer Systems*, 3(1):63–75, 1985.

[4] Flyxion. *Order and Obfuscation: Abstraction, Layering, and the Architecture of Computation*. Unpublished manuscript, 2026.

[5] J. Gray and A. Reuter. *Transaction Processing: Concepts and Techniques*. Morgan Kaufmann, 1993.

[6] L. Lamport. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7):558–565, 1978.

[7] L. Moreau and P. Missier, editors. *PROV-DM: The PROV Data Model*. W3C Recommendation, 30 April 2013.